LANCE

Where the project stands

State report as of 3 November 2026

LANCE project

Written by specialist strands, independently reviewed and synthesised

Last revision: 2026-11-03 10:30 (Europe/Brussels, UTC+01:00)

Documentation state as of 3 November 2026; no new experiment is reported

Abstract

LANCE is a platform for the security audit of authorised connected-object networks, developed in the context of the UMONS work of the NATO SPS G6259 programme. It asks one measurable question : *when does a language model find something that a competent automation cannot find, given the same tools, the same evidence and the same budgets ?*

This report answers a reading question : where does the project stand today ? Its first pages give the answer as a whole — what the system does, where the model actually decides, which three policies are compared, and what blocks the conclusion (Section 1). Section 2 lists what exists, what it guarantees and what it does not, with the measures of the period ; Section 3 states what can and cannot be asserted, and the three decisions that govern what follows.

The project counts a result only where no one-line rule settles the case and reasoning over several observations does. Until such situations are written and filtered, nothing else counts. Section 1 states that criterion, the three decisions that follow from it, and what would cross each blockage, with the test that would show it crossed.

The position is unchanged since the previous report of 30 October 2026 : the system is built, instrumented and documented, and its chain of evidence is sound. The scientific question remains open. No superiority of the model is established, no confidence interval can be computed, and the obstacle is no longer the computation but the absence of independent trials to compute on.

The document consistently distinguishes what is implemented, what is measured and what remains proposed. No new experiment is reported : the measurements cited are those already recorded, with their date. The limits of each component are explicit, including those of the sandbox, whose guard is not an exhaustive native-code sandbox.

Contents

1 Where the project stands **2**

2 What is built, and what is measured **6**

3 What remains **7**

4 Conclusion **8**

1 Where the project stands

This section answers the question of the report before the evidence arrives. It holds in four pages : what an audit does and who decides at each step (Figure 1), which three policies are compared and what is missing for the comparison to be fair (Figure 2), what blocks the conclusion and how the project got there (Table 1). The two sections that follow are the proof.

1.1 The project in one paragraph

A security auditor takes a network it has never seen, finds what is wrong with it, proves each finding rather than guessing it, and writes a report an operator can act on. LANCE does this with a language model in the loop. The programme asks a precise question : *the model earns its place only where a competent script would fail — does that situation exist, and can it be measured ?*

The answer requires three things, and the report follows that order. A system whose behaviour is traceable, so that a result can be attributed to a cause (Figure 1). A comparison in which the only thing that changes is the reasoning capacity, not the information or the tools (Figure 2). Trials where the two candidates genuinely differ, in the sense of Figure 3. The first exists, the second is half built, the third does not yet exist. That is the whole state of the project.

1.2 What an audit does, and where the model decides

An audit runs in six phases. Each phase produces one document that the next one reads, and nothing else crosses between them. Figure 1 is the map to keep in mind for the rest of the report : it shows the six phases, what each produces, and — the point that matters — *who* decides at each step.



Figure 1 – An audit in six phases, and where the model's freedom actually sits. The model decides at phases 2, 3, 4 and 5 : which probes to run, which weaknesses to hypothesise, how to test each one, and how far to push an intrusion. What the code decides is what becomes verifiable : phase 1 projects its evidence file from the graph tools and refuses a submission that misses a device. When the model's deliverable at phase 5 is missing or invalid, the code writes the assessment from the tool ledger, and a completed phase never certifies that the objectives were reached. Phase 6 assembles the report from section sheets, with an agent that declares no tool and runs a single turn. The score is produced elsewhere, by an offline evaluator that recounts the log against an oracle fixed before the run.

The log is the source, not the model's conclusion. Every tool call is archived with a reference, a refusal or an exception included. A finding that cites a piece of evidence which does not appear in the log is rejected. This is what makes a result attributable : the report states what happened, not what the policy believed happened.

The oracle precedes the run. The expected answer is fixed before the policy runs, in a file the policy never receives. A finding that contradicts the oracle invalidates the trial instead of correcting it. This precedence is what prevents the answer from being adjusted after the fact.

The score is recomputed, never taken on trust. The SSH, MQTT, API and V0 evaluators receive only the briefing, the oracle, the journal and the final decision, and they refuse a decision that cites an observation absent from the journal. The benchmark score of a pipeline run does not perform that check ; the confirmed metrics it would need are not emitted by the harnesses today.

Where this lives. Phase registry `src/agent/phases/registry.py` ; governance boundary `src/agent/core/executor.py` ; evaluators under `src/benchmark/` and `benchmarks/experiments/`.

1.3 The three policies compared on the same scenario

The programme compares three ways of answering the same question on the same scenario. They receive the same targets and are judged by the same evaluator ; what differs is how the answer is produced. The platform also offers four execution modes — `lance` (the full pipeline, with a model), `rules` (analysis and verification only — scanner and aggregation, with no model at all), `vanilla` (a model exploring an address range) and `scripted` (versioned procedures) ; only `lance` is scored against ground truth ; the others are not.

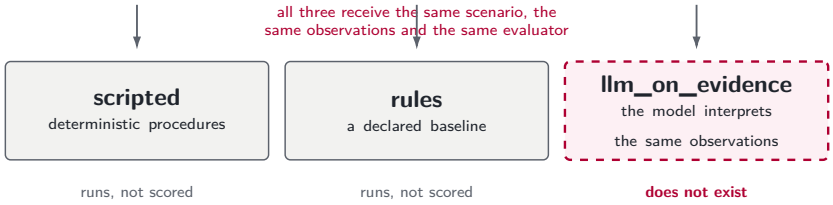


Figure 2 – The three policies and what is missing. The third, `llm_on_evidence`, would give the model the observations the rules already receive and nothing else. It does not exist, and without it the comparison measures verification rather than interpretation.

Why the third policy is the one that matters. An exploration where the model roams a network freely (`vanilla`) changes too much at once : the targets chosen, the order of the probes and the interpretation all differ, so a better score cannot be attributed to reasoning. A policy that receives the scanner's observations and must produce findings backed by a citation of those observations isolates exactly one variable : *who interprets*. That is the question the programme asked.

What currently favours the rules. Four asymmetries separate the declared baseline from the model. The rules arm executes a deterministic probe plan for each finding, while the model must find its own sequence. In the `rules` mode the analysis phase runs without a model at all, so no hypothesis is lost in translation. The scanner's findings feed both aggregations, but there they are the whole of the rules arm rather than a supplement to it. And on the compact path the pipeline writes the scanner's findings over the model's per-device sheet, an asymmetry that runs the other way. Until these are removed or mirrored, an equal score would say nothing.

1.4 What blocks the conclusion

The obstacle is not the measurement machinery, which is now sound, but the material it would need. The material it would need is a trial that a one-line rule does not cross ; the episode of 30 October 2026 is the only one attempted, and it was rejected in an hour.

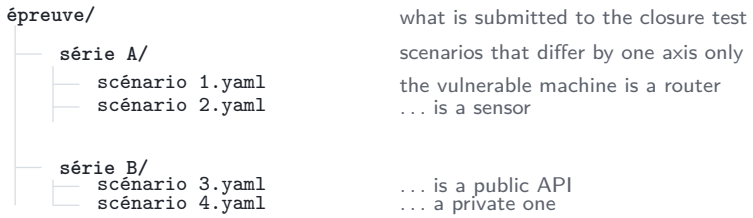


Figure 3 – The three units of the benchmark, lowest to highest. A **scenario** is one deployment that gets audited. A **series** holds the scenarios that differ by a single axis — here, only the kind of vulnerable machine changes. A **trial** is what is submitted to the closure test : it holds one or several series and counts as *one* observation, so the four scenarios above are a single data point, not four. Earlier reports said *family* ; the word is retired because it named the grouping and never what the grouping had to prove.

What the closure test is. Before building a trial, one asks whether a rule that solves the case can be written in a single line. If it can, the trial measures the omission of a declaration rather than a limit of interpretation, and it is not built. The test is cheap ; running it before writing eight fixtures is the change of method this month produced.

Why this is a blockage and not a delay. The inference tool was repaired on 30 October : it now computes a one-sided p-value against the pre-registered threshold, refuses to emit a figure below three trials, and requires five to decide. It is therefore correct, and idle : there is nothing to compute on. Building more trials of the same kind would not change the arithmetic.

1.5 What must be decided

The blockage is not technical, so the way out is not more machine time. Three choices govern what follows, and none of them is taken yet.

1. Trials. Build trials that a one-line rule does not cross, applying the filter that one hour of observation established, or publish a bounded methodological result and declare it as such. The filter matters more than the count : a trial crossed by a generic rule measures nothing, and five of them measure nothing either.

2. The opening condition. Write the competent reference automation that D1+ names, or renounce explicitly the hypotheses that depend on it. Several trials remain undecidable until this is settled.

3. The comparable arm. Give the model the observations the rules already receive, or state that the comparison measures verification rather than interpretation. The second option is legitimate and much cheaper ; it is not the question the programme asked.

These three decisions together determine whether an inference can one day exist.

1.6 The state in one table

Table 1 – The state of the project on 2 November 2026, and where the proof is.

Question	Answer	**Proof**
Is an audit traceable ?	Yes. Every tool call is archived with a reference, bounded, and a piece of evidence that cannot be archived stops the run instead of leaving a silent gap. The guards are application-level.	**Table 2**
Does the harness run outside the laboratory ?	Yes, without broker and without a model, under a guard whose blind spots are declared. It instruments a contrast ; it measures nothing.	**Table 2**
Are the protocols' contracts sound ?	Yes. Briefing, private oracle and offline evaluator are separated, and the oracle is fixed before the run.	**Table 2**
Can results be delivered reproducibly ?	Yes. Push, validate, deploy to one of three instances ; no execution on a workstation. Single points of failure declared.	**Table 2**
Does the model beat the rules ?	**Unknown, and not yet measurable.**	**Section 2**
How much of the corpus needs interpretation ?	Two counts from two artefacts, never to be cited one for the other. Binary index : 282 entries by signature, 6 by logic (2.1 %). Interpretive audit : 243 decidable by a well-designed rule, 35 mixed, 4 chaining, 6 logic (84.4 %) — a sum over categories, not a field per entry.	**Section 2**
Is the measurement statistically valid ?	The tool is repaired and refuses an indefensible figure ; the trials that would give it data do not exist.	**Section 2**

1.7 How far the project goes

The state of each item — reached or not — is in the project's current state page (`objectifs/ETAT-CURANT.md`), which this checkpoint does not repeat. What that table cannot carry is the other half : *what would end each blockage*, as a criterion one can observe rather than a hope.

Three of these four depend on the first. A comparable arm, a written reference automation and a corpus built for the question are worth building only once a situation exists that a one-line rule does not settle ; before that, each of them measures the absence of that situation. **The first line is the only work that can start today.**

This is where the report stops. Everything up to a verified measurement is built ; the thing to be measured is not. The next result the project can legitimately produce is *a trial that survives its closure test*, not a better score — and it is the only line of the table whose proof needs no new agreement on method.

2 What is built, and what is measured

In brief. Nine components exist, each with a stated guarantee and a stated limit. None of them proves that a model does better than a script, which is the object of the programme. The detail is in `docs/` for the behaviour and in the archived reports for the implementation ; what follows is what a committee needs in order to know whether the instruments can be trusted.

Table 2 – What exists, what it guarantees, and what it does not.

Component	Guarantee	Limit
Execution boundary	every tool call is archived with a reference, bounded by budgets, refused outside the target network	application-level guards, not a system sandbox
Laboratory	a scenario deployed, configured and reset, then destroyed : a trial starts from the same state	one host, one container, one runner
Delivery	push, validate, deploy to one of three instances ; no execution on a workstation	no automatic rollback after a failed deployment
Audit pipeline	six phases, one document per phase, the model decides where interpretation is needed (Figure 1)	two scenarios on one laboratory are not mutually excluded
Evidence contracts	briefing, private oracle and offline evaluator separated ; the oracle is fixed before the run	four protocols only
Harnesses	one contract, one deterministic arm, one model arm, one evaluator per protocol	the H2 arms are offline ; other controllers need the laboratory
Sandbox	the deterministic arms run without broker, model or laboratory, under a guard whose blind spots are declared	it instruments a contrast, it measures nothing
Corpus	29 scenarios, 288 ground-truth entries	built around what signature rules find (Table 1)
Inference tool	one-sided p-value against the pre-registered threshold, no figure below three trials	it has nothing to compute on

2.1 What is measured, and what the measures say

The fidelity gate passes, partially. The V0 control, a task solvable without inference, succeeds on both arms three times out of three, with a saturating control : it is readable for a repeated defect, not for a rare one. The threshold frozen at 0.9 is computed separately by the gate and conditions nothing.

One trial was measured, and its gain was absorbed. H2, on 22 October 2026 : eight fixtures, 25 messages, two readable runs. The first ran the two arms of the time ; the reference run that replaced it rotates three arms over three repetitions, 72 arms. On the raw variants the model arm scores 0.5, the declared parser 0.0, and an arm reading by convention covers 8 of 8 — it absorbs the gain without declaring anything.

One trial was built and rejected in an hour. The vendor-register probe of 30 October resisted six recorded reviews and fell to a rule that reads the effect of a write on the neighbouring registers. The filter that would have caught it costs minutes : if the rule settling the case can be written in one line, a script will have it.

The inference is repaired and idle. The tool now computes a one-sided p-value against the threshold, refuses to emit a figure below three trials and requires five to decide. No set of five independent trials exists, so the arithmetic has nothing to work on.

The exploratory pilot published three pairs, not twelve. The reconciliation of 16 October plans twelve pairs ; seven were executed under a freeze, three were published as comparable, five were incomparable, and the campaign is not admissible.

2.2 The corpus, and why it cannot answer the question

Table 3 – Two counts, from two artefacts, never to be cited one for the other.

Reading	Decidable by a rule	Not decidable	Total
Mechanical (binary `type` of each entry)	282 by signature	6 by logic (2.1 %)	288
Interpretive (category judged as a whole)	243 (84.4 %)	35 mixed, 4 chaining, 6 logic	288

Both are recomputed from `benchmarks/labels/derived\_index.json` for the mechanical reading and from the coverage audit of 12 October 2026 for the interpretive one, which totals categories rather than entries. A duel on this corpus would rank rules against rules, which is exactly what H2 observed.

3 What remains

3.1 What the project can assert, and what it cannot

Four assertions are defensible, each with its limit. An audit produces verifiable traces : every tool call is archived and referenced, and the run stops when its evidence log cannot be written — but the guards are application-level. A harness can run outside the laboratory, without broker or model, under a guard that declares its blind spots — but it instruments and measures nothing. A third script arm absorbs a gain attributed to the model on the most documented trial — but that holds for eight fixtures and one property. The corpus is mostly detectable by signature, which makes a null difference expected by construction — but that concerns this corpus, not the situations the programme targets.

Must not be written as established : the superiority of the model ; a significant gap, since no independent trial exists to compute one ; generality, H2 covering one schema, one question and one model ; a cost gain ; the validity of the V0 threshold, which the gate does not apply ; and the integrity of the corpus, the official healthy control being absent from the catalogue.

3.2 The blockage, and what would cross it

The obstacle is not the measurement machinery, which is now sound, but the material it would need : situations where the answer is not settled by a rule. Table 4 states, for each blockage, the observable test that would show it crossed. Three of its four rows depend on the first : a comparable arm, a written reference automation and a corpus built for the question measure the absence of that situation until one exists. **The first row is the only work that can start today.**

Table 4 – Each blockage, what would cross it, and how one would know. No term here needs the glossary : the table is meant to be read before it.

Blockage	What would cross it	How one would know
No situation a rule cannot decide	a case where no one-line rule settles the answer	the closure script runs and refuses to admit it
No comparable arm	the model interpreting the very observations the rules receive	the same scenario scored on both, by the same evaluator
The competent reference is a doctrine	a written reference automation standing in for it	it passes a trial that the model also passes
Corpus built for rules	scenarios where a well-written rule can only abstain	the interpretive share rises, recomputed entry by entry

3.3 The three decisions

1. Trials. Build trials that a one-line rule does not cross, applying the filter that one hour of observation established, or publish a bounded methodological result and declare it as such.

2. The opening condition. Write the competent reference automation that D1+ names, or renounce explicitly the hypotheses that depend on it.

3. The comparable arm. Give the model the observations the rules already receive, or state that the comparison measures verification rather than interpretation.

These three decisions together determine whether an inference can one day exist. None of them is technical in the narrow sense : all of them concern what one accepts not to be able to say.

4 Conclusion

The system is built, instrumented and documented. Its chain of evidence is sound : a separate oracle, an evaluator that recounts, a log that does not forgive a lost piece of evidence, a safeguard that declares its blind spots. The defects found in this report were found by reading what the code does, not what it claims, and that is the best evidence that the verification discipline works.

The scientific question remains open. The programme asked when a model does better than a script with identical tools ; the available data do not answer it, and the current arrangement would not allow an answer even with more machine time. The next result the project can legitimately produce is *a trial that survives its closure test*, not a better score.